



PF2UML: A Tool for Problem-Oriented Requirements Modeling and Transformation

Hongbin Xiao
Guangxi Key Lab of Multi-source
Information Mining & Security
Guangxi Normal University
Guilin, China
hongbinxiao@stu.gxnu.edu.cn

Zhi Li*
Guangxi Key Lab of Multi-source
Information Mining & Security
Guangxi Normal University
Guilin, China
zhili@gxnu.edu.cn

Yilong Yang
School of Software
Beihang University
Beijing, China
yilongyang@buaa.edu.cn

Fei Tang
Huawei Technologies Co., Ltd
Shenzhen, China
fei.tang@huawei.com

Dongming Jin
School of Computer Science
Peking University
Beijing, China
dmjin@stu.pku.edu.cn

Abstract

Requirements engineering is a critical phase in ensuring the success of software development projects. However, transitioning from requirement analysis to system design often encounters challenges, including ambiguity in requirement expression, understanding deviations, and inefficiencies caused by manual operations. To address these issues, we propose PF2UML, a tool designed to bridge the gap between requirements analysis and system design. PF2UML offers three core features: (1) support for problem-oriented requirements modeling based on Problem Frames theory, (2) automated transformation of requirement models into UML diagrams. Experimental evaluations demonstrate that PF2UML significantly outperforms traditional manual methods, achieving improvements of 34% in time efficiency for requirements modeling, significantly improving the efficiency of model transformation, and effectively ensuring the consistency from requirements modeling to system design. Moreover, six case studies reveal that PF2UML achieves more than 10 times the efficiency of manual processes in complex scenarios.

The tool is publicly available at <https://github.com/HongbinXiao/PF2UML>, with a demonstration video accessible at <https://youtu.be/jB-G6LqmTdw>.

CCS Concepts

• **Software and its engineering** → **Requirements analysis; Model-driven software engineering; Software design engineering.**

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

FSE Companion '25, June 23–28, 2025, Trondheim, Norway

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1276-0/25/06

<https://doi.org/10.1145/3696630.3728596>

Keywords

Requirements Engineering, Problem Frames, UML, MDE

ACM Reference Format:

Hongbin Xiao, Zhi Li, Yilong Yang, Fei Tang, and Dongming Jin. 2025. PF2UML: A Tool for Problem-Oriented Requirements Modeling and Transformation. In *33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion '25)*, June 23–28, 2025, Trondheim, Norway. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3696630.3728596>

1 Introduction

Requirements engineering is a critical phase in software development. However, in practical applications, the transition process from requirements analysis to system design often faces many challenges[14, 21]. These challenges stem from the ambiguity in the expression of user requirements, developers' bias in understanding requirements, and the limitations of traditional methods in requirements modeling and design conversion. Consequently, this process frequently leads to problems such as low efficiency, implementation difficulties, and design errors[2, 13, 19].

To address these fundamental challenges, structured approaches to requirements analysis have emerged. Michael Jackson's Problem Frames theory [11] provides one such structured approach to analyzing and describing software problems. Problem Frames decompose complex problems into known problem types (such as information display, state transformation, etc.), and provide a framework to describe problem domains, the machine, and the interfaces between them. This method is particularly suitable for capturing and analyzing system requirements because it focuses on the problem itself rather than the solution, helping developers gain a deeper understanding of the problem's essence.

While Problem Frames offer a structured way to understand requirements, transforming these requirements into design models remains essential. Converting requirements to UML models is crucial for software development for three reasons: First, UML, as an industry-standard modeling language, provides clear visual representation; second, UML models can directly guide system implementation; finally, UML models can be used to verify whether

the system meets the original requirements. However, this conversion process faces numerous challenges, including the complexity and ambiguity of requirements, as well as requirement evolution.

To overcome these conversion challenges, researchers have developed various tools and methods to enhance requirements engineering through automation and modeling. Despite these efforts, existing tools still struggle with accurately capturing and structuring user requirements, reducing ambiguity, and efficiently converting requirements into UML models. They also face significant challenges in ensuring consistency and traceability between requirements and design, often resulting in errors and inefficiencies[1, 4]. This gap between structured requirements understanding and effective design modeling represents a critical area for improvement in software engineering practice.

In this paper, we present PF2UML: a tool for supporting the automatic generation of UML models from problem frames requirements models. PF2UML offers two main features:

1) **Problem-oriented requirements modeling and analysis:** PF2UML provides a visual platform based on Problem Frames theory [11]. This platform enables comprehensive modeling and validation of system requirements and improves communication among stakeholders.

2) **Automatic generation of UML models:** The tool automates the transformation of problem frames models into UML models[24], such as class diagrams and use case diagrams, providing a clear blueprint for system design.

The remainder of this paper is organized as follows: Section 2 discusses related tools. Section 3 presents the features of PF2UML. Section 4 presents the evaluation results on the four case studies. Section 5 concludes this paper.

2 RELATED TOOLS

Based on the functional characteristics of PF2UML, tools closely related to it can be categorized into two types:

Problem-Oriented Requirements Modeling Tools: Several tools based on the Problem Frames (PF) approach have been developed to support requirements modeling, problem decomposition, and complexity analysis. For instance, PF4MD supports microservice decomposition by refining causal domains[15], NL2PD extracts requirements from natural language to generate problem diagrams[5], Trace4PF facilitates problem diagram decomposition through an interactive interface[9], and ZOOM4PF integrates UML and SysML to refine problem domain descriptions[23].

However, these tools face limitations in handling complex problems, achieving high levels of automation, and integrating with UML models. PF2UML addresses these challenges by enhancing the decomposition of complex problems, improving consistency verification between problem diagrams and UML models, and providing automated modeling support. These advancements significantly improve the practicality of problem-oriented requirements modeling tools.

UML Modeling Tools: Recent advances in UML modeling tools have focused on supporting system requirements analysis. For example, the LUCAM-based tool proposed by Miranda et al. automatically generates use case and class diagrams from functional requirements, improving model consistency and reducing manual

effort [16]. Similarly, the tool developed by Ahmad et al. generates test cases from activity diagrams to validate the alignment between requirements models and system functionality [3]. Manual modeling tools such as MagicDraw and Papyrus enhance user interaction, improving modeling flexibility and efficiency [17]. Deepthimahanti and Babar's domain-independent UMGAR tool employs NLP techniques and syntactic refactoring rules to handle complex requirements[8]. Ramackers et al. envisioned "Prose to Prototype" (P2P), an AI-powered approach with human-in-the-loop interaction to transform natural language into UML models[18].

However, automated tools often lack the flexibility to handle complex and evolving requirements, while manual tools rely heavily on user expertise, leading to inefficiencies. Moreover, existing tools struggle with consistency verification and real-time system modeling, limiting their ability to fully support complex systems. In contrast, PF2UML automatically generates UML models aligned with requirements, enhancing traceability, improving consistency verification, and reducing manual intervention. This approach not only increases the accuracy of requirements analysis, but also strengthens support for designing complex systems.

3 PF2UML FEATURES

3.1 Overview

The PF2UML tool takes system requirements as input, initially constructing a problem-oriented requirements model through a collaborative modeling approach. Subsequently, the tool converts this model into a lightweight UML model. This section provides a detailed explanation of the three main stages of PF2UML: requirements modeling and analysis, generating UML models from problem models, and requirements decomposition and refinement. The entire process is illustrated in Figure 1.

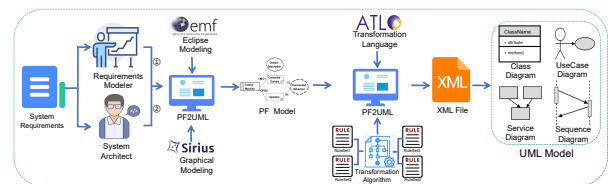


Figure 1: Overview of the Main Process of PF2UML

3.2 Requirements Modeling and Analysis

The PF2UML tool adopts the Problem Frames approach to enable efficient modeling and analysis of complex system requirements in a collaborative modeling environment[7, 11]. Its core process is based on role collaboration, where requirements modelers are responsible for constructing problem-oriented requirements model frameworks[10], while system architects complement the process by adding object-oriented details. Leveraging the graphical modeling support provided by the Eclipse Modeling Framework[20] and Sirius[22], PF2UML creates an intuitive collaborative platform that allows users to construct problem-oriented requirements models using graphical tools, explicitly defining the relationships between system requirements and problem domains. Based on Jackson's

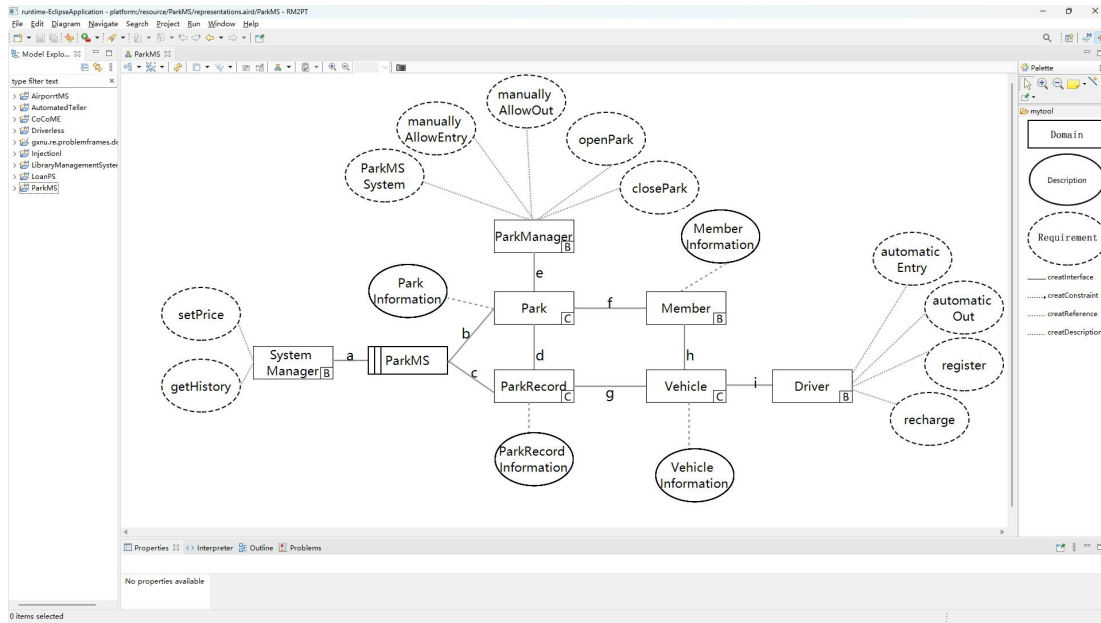


Figure 2: Problem-Oriented Collaborative Requirements Modeling Platform

Problem Frames theory, the tool decomposes system requirements into multiple problem domains and their shared phenomena, emphasizing the structured representation of requirement interactions and constraints to ensure that the models accurately reflect the actual system requirements and their environment.

During the modeling process in PF2UML, modelers can refine and supplement the details of the models by double-clicking on individual elements, utilizing Problem Frames theory. As illustrated in Figure 2:

Problem Domain: Users can specify the domain type (e.g., control domain, command domain), whether it corresponds to a class in the class diagram, and its subdomain information.

Description Domain: Users can define their attributes, such as characteristics and states.

Requirement Domain: Sub-requirements and their relationships can be added, along with service diagram information.

Interface: Six types of shared phenomena (e.g., events, commands, data) can be defined.

Requirements Reference and Constraint: Users can add referenced requirements and specify detailed constraints.

Through these operations, the PF2UML tool systematically decomposes and manages complex system requirements, providing clear and structured support for collaborative modeling.

3.3 Requirements Model Transformation

PF2UML offers automated requirements model transformation capabilities, relying on four sets of transformation rules defined using the Atlas Transformation Language[12]. These rule sets collectively form the transformation algorithm. The specific workflow, as illustrated in Figure 1, involves taking the collaboratively modeled Problem Frames Model as input, applying the transformation algorithm to convert it into an XML file, and finally parsing the XML

file to generate lightweight UML artifacts[6, 24], such as conceptual class diagrams, use case diagrams, service diagrams, and system sequence diagrams.

4 EVALUATION

To evaluate the effectiveness and efficiency of the PF2UML tool, we conducted a series of case studies and comparative experiments. This section describes in detail the evaluation methodology, case selection criteria, experimental setup, and results.

In this paper, we use "complexity" to specifically refer to the structural complexity of requirements models, including the number of problem domains (PD), domain dependencies (DD), requirements (RD), shared phenomena (SP), and domain attributes (DA). These metrics collectively reflect the scale and internal connectivity of the requirements model, which differs from simple scalability or implementation difficulty. Highly complex models typically feature more interaction interfaces and information flows, requiring more refined problem decomposition and stricter consistency management. By quantifying these structural characteristics, we can objectively assess the inherent complexity of different requirement scenarios, providing standardized benchmarks for tool performance testing.

4.1 Case Studies

To evaluate the PF2UML tool, we analyzed six open-source systems¹: Airport Maintenance (AirMS), ATM, Supermarket (CoCoME), Library Management (LibMS), Loan Processing (LoanPS), and Parking Management (ParkingPS). These diverse cases span various domains, and their PF model complexities are summarized in Table 1. To evaluate the functionality of the PF2UML tool, experiments

¹<https://ai4se.com/casestudy>

Requirements Model

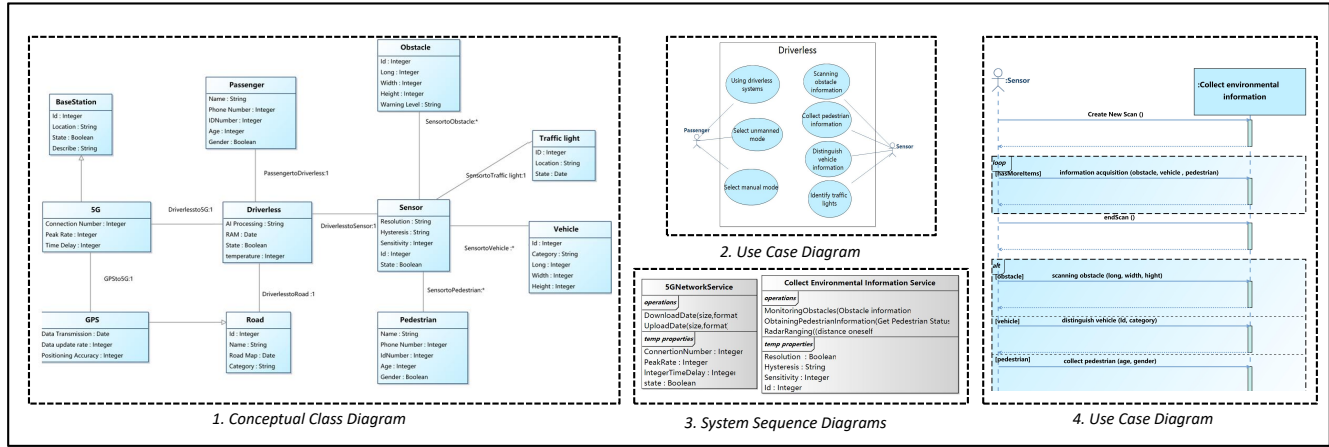


Figure 3: Requirement Refinement and Decomposition: Transition from System Requirements to Software Requirements

Table 1: Complexity of PF Models across Six Case Studies

Case Study	PD	DD	RD	SP	DA
ATM	11	3	6	54	10
CoCoME	16	13	16	143	34
LibMS	15	11	22	175	48
LoanPS	14	8	11	89	35
APMS	9	4	8	46	21
ParkMS	8	4	11	87	19
Sum	73	43	74	594	167

* PD represents Problem Domain, DD represents Domain Description, RD represents Requirement Domain, SP represents Shared Phenomena within interfaces, DA represents Domain Attributes

were conducted on these six case studies, with the results and processes documented on GitHub² and YouTube³.

4.2 Evaluation Results

In the experiments, we selected six case studies and performed requirements modeling and subsequent model transformation for each case using both manual methods and the PF2UML tool. All UML models automatically generated by PF2UML were verified through RM2PT [25]. During our evaluation process, RM2PT validated the static structure of the models.

As shown in the Table2, PF2UML reduced the average requirement modeling time by 34% (from 62.6 minutes to 41.33 minutes) and drastically accelerated the model transformation process, reducing the time from 56.5 minutes to just 1.07 seconds. For instance, the largest case study (CoCoME) was completed in 3.49 seconds, while the smallest (APMS) took only 0.13 seconds. These results highlight PF2UML's ability to handle varying complexities efficiently, showcasing its potential for significantly improving the speed of requirements modeling and transformation.

²<https://github.com/Hongbin-Xiao/PF2UML>

³<https://www.youtube.com/playlist?list=PLI9l-9orgj5h-8lWDtLpVUym8uoCKYVC>

Table 2: Time Comparison of Requirement Modeling and Model Transformation

Tasks	ATM	CoCoME	LibMS	LoanPS	APMS	ParkMS	AVG
RM (Manual)	29m	115m	80m	58m	53m	41m	62.6m
RM (PF2UML)	20m	70m	55m	35m	41m	27m	41.33m
MT (Manual)	34.5m	79.5m	93.1m	55m	30.7m	46.2m	56.5m
MT (PF2UML)	0.43s	3.49s	0.72s	1.33s	0.13s	0.33s	1.07s

* RM represents Problem-Oriented **Requirement Modeling**, and MT represents **Model Transformation** to UML, m represents minutes, s represents seconds

5 CONCLUSION AND FUTURE WORK

PF2UML is a tool that improves the transition from requirements analysis to system design by supporting problem-oriented modeling, generating UML diagrams automatically, and refining system requirements into software requirements. Case studies show it significantly enhances efficiency and reduces manual effort.

Future plans include supporting more modeling languages like Systems Modeling Language (SysML), improving handling of complex requirements, integrating with prototype generation tools to streamline development workflows, and conducting broader tool ecosystem research that will include more comprehensive comparative analyses.

6 ACKNOWLEDGMENT

This work was supported by the National Science Foundation of China (Grant No.62362006), the Guangxi Science and Technology Program (Grant No.GuiKeAB24010343), and the Innovation Project of Guangxi Graduate Education (Grant No.XYCBZ2024024).

References

- [1] Syed Manzar Abbas, Khubaib Amjad Alam, Umer Iqbal, and Sahar Ajmal. 2019. Quality factors enhancement of requirement engineering: A systematic literature review. In *2019 International Conference on Frontiers of Information Technology (FIT)*. IEEE, 13–135.
- [2] Rosmiza Wahida Abdullah, Sabrina Ahmad, Siti Azirah Asmai, Lee Seok-Won, and Zarina Mat Zain. 2021. Research efforts and challenges in crowd-based requirements engineering: a review. *International Journal of Advanced Computer Science and Applications* 12, 9 (2021).
- [3] Tanwir Ahmad, Junaid Iqbal, Adnan Ashraf, Dragos Truscan, and Ivan Porres. 2019. Model-based testing using UML activity diagrams: A systematic mapping study. *Computer Science Review* 33 (2019), 98–112.
- [4] Oscar Benito-Pacheco, Hugo Vega-Huerta, Percy De-La-Cruz-VdV, Ernesto Cancho-Rodriguez, Ronald Melgarejo-Solis, Jorge Pantoja-Collantes, and Javier Cabrera-Diaz. 2024. Challenges of Requirements Engineering in Agile Projects: A Systematic Review. *International Journal of Emerging Technology and Advanced Engineering* 14, 1 (January 2024), 13–21. https://doi.org/10.46338/ijetae0124_03
- [5] Xuan Chen, Hongbin Xiao, Yajun Deng, and Zhi Li. 2023. NL2PD: A Tool for Problem Diagram Generation from Requirements in Natural Language. In *2023 IEEE 31st International Requirements Engineering Conference (RE)*. IEEE, 361–362.
- [6] Zhenbang Chen, Zhiming Liu, Anders P Ravn, Volker Stolz, and Naijun Zhan. 2009. Refinement and verification in component-based model-driven design. *Science of Computer Programming* 74, 4 (2009), 168–196.
- [7] Karl Cox, Jon G Hall, and Lucia Rapanotti. 2005. A roadmap of problem frames research. *Information and Software Technology* 47, 14 (2005), 891–902.
- [8] Deva Kumar Deeptimahanti and Muhammad Ali Babar. 2009. An automated tool for generating UML models from natural language requirements. In *2009 IEEE/ACM International Conference on Automated Software Engineering*. IEEE, 680–682.
- [9] Yajun Deng, Zhi Li, and Hongbin Xiao. 2022. Trace4PF: A tool for Automated Decomposition of Problem Diagrams with Traceability.. In *SEKE*. 473–474.
- [10] Jon Hall, Lucia Rapanotti, and Michael Jackson. 2008. Problem oriented software engineering: Solving the package router control problem. *IEEE Transactions on Software Engineering* 34, 2 (2008), 226–241.
- [11] Michael Jackson. 2000. *Problem Frames: Analyzing and structuring software development problems*. Addison-Wesley Longman Publishing Co., Inc.
- [12] Frédéric Jouault and Ivan Kurtev. 2005. Transforming models with ATL. In *International Conference on Model Driven Engineering Languages and Systems*. Springer, 128–138.
- [13] Rashidah Kasauli, Grischia Liebel, Eric Knauss, Swathi Gopakumar, and Benjamin Kanagwa. 2017. Requirements engineering challenges in large-scale agile system development. In *2017 IEEE 25th International Requirements Engineering Conference (RE)*. IEEE, 352–361.
- [14] Phillip A Laplante and Mohamad Kassab. 2022. *Requirements engineering for software and systems*. Auerbach Publications.
- [15] Yulong Li, Zhi Li, Yitao Bu, Hongbin Xiao, and Yajun Deng. 2023. PF4MD: A Microservice Decomposition Tool Combining Problem Frames. In *2023 IEEE 31st International Requirements Engineering Conference (RE)*. IEEE, 359–360.
- [16] Márcio Assis Miranda, Marcos Guilherme Ribeiro, Humberto Torres Marques-Neto, and Mark Alan Junho Song. 2018. Domain-specific language for automatic generation of UML models. *IET Software* 12, 2 (2018), 129–135.
- [17] Elena Planas and Jordi Cabot. 2020. How are UML class diagrams built in practice? A usability study of two UML tools: Magicdraw and Papyrus. *Computer Standards & Interfaces* 67 (2020), 103363.
- [18] Guus J Ramackers, Pepijn P Griffioen, Martijn BJ Schouten, and Michel RV Chaudron. 2021. From prose to prototype: synthesising executable UML models from natural language. In *2021 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*. IEEE, 380–389.
- [19] Aqsa Rasheed, Bushra Zafar, Tehmina Shehryar, Naila Aiman Aslam, Muhammad Sajid, Nouman Ali, Saadat Hanif Dar, and Samina Khalid. 2021. Requirement engineering challenges in agile software development. *Mathematical Problems in Engineering* 2021, 1 (2021), 6696695.
- [20] Dave Steinberg, Frank Budinsky, Ed Merks, and Marcelo Paternostro. 2008. *EMF: eclipse modeling framework*. Pearson Education.
- [21] Axel Van Lamsweerde. 2008. Requirements engineering: from craft to discipline. In *Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of software engineering*. 238–249.
- [22] Vladimir Viyović, Mirjam Maksimović, and Branko Perišić. 2014. Sirius: A rapid development of DSM graphical editor. In *IEEE 18th International Conference on Intelligent Engineering Systems INES 2014*. IEEE, 233–238.
- [23] Shangfeng Wei, Zhi Li, Yilong Yang, and Hongbin Xiao. 2021. Zoom4pf: A tool for refining static and dynamic domain descriptions in problem frames. In *2021 IEEE 29th International Requirements Engineering Conference (RE)*. IEEE, 414–415.
- [24] Yilong Yang, Xiaoshan Li, Wei Ke, and Zhiming Liu. 2019. Automated prototype generation from formal requirements model. *IEEE Transactions on Reliability* 69, 2 (2019), 632–656.
- [25] Yilong Yang, Xiaoshan Li, Zhiming Liu, and Wei Ke. 2019. RM2PT: a tool for automated prototype generation from requirements model. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, 59–62.